



전자전기컴퓨터설계실험II

실험 전 보고서

LAB1

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 9월 11일
대상	조합회로 실습
검증 환경	Vivado XSim 2026.1 / VS Code VaporView

1. 목적과 관련 이론

실험 목적

개별 회로 10개를 VS Code에서 먼저 검증하고 Vivado와 보드에 확인한다.

관련 이론

-FPGA와 ASIC의 차이

FPGA는 Field Programmable Gate Array이고 제조 후 회로 변경이 가능한 반도체 IC이다. ASIC는 Application-Specific Integrated Circuit이고 특정 용도를 위해 설계·제조한 전용 반도체다. FPGA는 ASIC에 비해 성능 및 대량생산 단가에서 단점이 존재하지만 제조 후 재구성성이 가능하고 개발 기간과 초기 개발 비용이 저렴하다는 장점이 있다.

-FPGA를 사용하는 두 가지 이유

(반도체 공정과 관련지어 조사)

1. 설계 변경 시 반도체를 다시 제조하지 않아도 된다. ASIC는 설계된 회로를 실제 실리콘 위에 구현하기 위해 웨이퍼 공정을 거치고 제조 후 회로 구조가 고정된다. 따라서 설계 오류나 기능 변경을 할 경우 수정된 설계로 다시 제조해야 할 수 있다. 반면 FPGA는 제조 이후에 내부 논리와 배선 구조를 재구성할 수 있어 회로를 변경할 수 있다. 따라서 별도의 추가 공정 없이 설계를 수정할 수 있어 개발 및 검증 과정에서 유리하다.

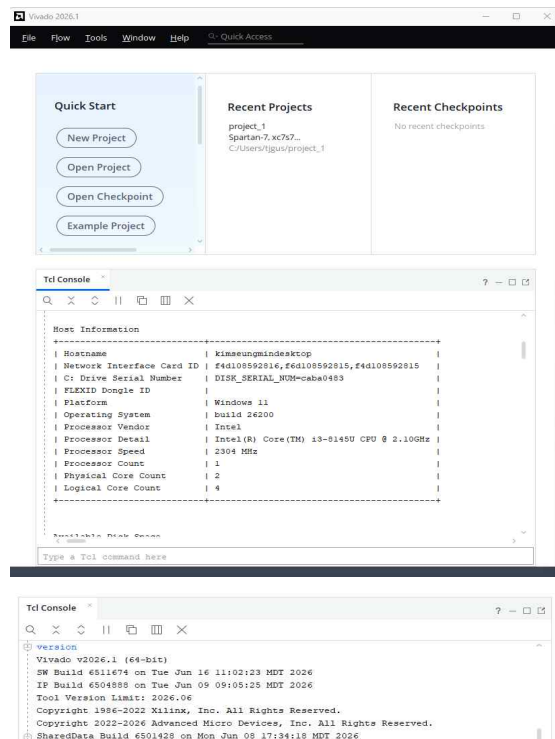
2. FPGA는 ASIC 제작에서 발생하는 높은 초기 제조 비용을 줄일 수 있다. ASIC는 회로 설계 이후 실제 반도체 생산을 위해 포토마스크 제작과 웨이퍼 공정을 거쳐야 하므로 높은 비용이 발생한다. 반면 FPGA는 제조업체가 미리 제작한 범용 반도체 칩의 내부 논리와 배선 구조를 프로그래밍하기 때문에 새로운 회로를 설계할 때마다 별도의 추가 공정이 필요없다. 따라서 ASIC보다 초기 개발 비용을 절약할 수 있다.

실행 방법

1. 공개 템플릿을 복제하고 새 창에서 연다.
2. RTL, TB, XDC를 직접 작성하고 네 입력 조합을 검사한다.
3. 같은 소스를 Vivado에 등록하고 다시 시뮬레이션한다.
4. Spartan-7용 비트스트림을 생성한다.

실행 환경

- Vivado 2026.1 설치와 실행 확인



- GitHub 계정 생성 및 주소 기재

계정: thechef030

주소: <https://github.com/thechef030>

공통된 도구 버전

icarus verilog: version 13.0

git: version 2.55.0.windows.5

python: python 3.13.15

vscode: 1.137.0

2. 실험 1 AND OR XOR 논리 게이트

목적과 입출력 진리표

두 개의 1비트 입력 a, b에 대해 $x = \text{AND}$, $y = \text{OR}$, $z = \text{XOR}$ 을 비교한다. AND, OR, XOR 논리 연산을 수행하는 조합 논리 회로를 설계하고 시뮬레이션한다. AND는 입력이 모두 1, OR은 하나 이상이 1, XOR은 서로 다른 입력일 때 1이다. RTL 코드는 다음과 같다.

```
assign x = a & b;
```

```
assign y = a | b;
```

```
assign z = a ^ b;
```

RTL 코드에 따라서 x는 AND, y는 OR, z는 XOR에 대응하여 구현된다.

예상값은 다음과 같다.

a	b	x AND	y OR	z XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

RTL 코드는 다음과 같다.

```
assign x = a & b;
```

```
assign y = a | b;
```

```
assign z = a ^ b;
```

RTL 코드에 따라서 x는 AND, y는 OR, z는 XOR에 대응하여 구현된다.

템플릿경로, 실행 task, 코드커밋

- 템플릿 경로: https://github.com/thechef030/lab1_01_logic_gates/blob/main/sim/tb_logic_gate_modern.sv

- 실행 task: 01 Check tools
02 Simulate
03 Open waveform

-코드커밋: 6f0b788 (HEAD -> main) LAB1

logic gate simulation

https://github.com/thechef030/lab1_01_logi_c_gates/commit/6f0b78817d4c6c2f2a89c0bb2f64c448eecf8b20

테스트벤치 코드

테스트벤치의 네입력은 00, 01 ,10, 11이다. 기댓값은 다음과 같다.

	a	b	x	y	z
0~10 (ns)	0	0	0	0	0
10~20 (ns)	0	1	0	1	1
20~30 (ns)	1	0	0	1	1
30~40 (ns)	1	1	1	1	0

테스트벤치의 네 입력을 순차적으로 인가한다. 각 입력 조합은 10ns 동안 유지되며 총 40ns 동안 모든 입력 조합을 검사한다. 입력값의 변화는 40ns에 끝나기 때문에 테스트벤치는 40ns에 종료된다.

```
`timescale 1ns/1ps
module tb_logic_gate_modern;
    reg a,b; wire x,y,z; logic_gate dut(a,b,x,y,z);
    integer n,checked=0;
    reg [2:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_logic_gate_modern);

        For(n=0;n<4;n=n+1) begin
            {a,b}=n;
            expected=~((a & b),(a | b),(a ^ b));
            #10;
            if ({x,y,z} != expected)
                $fatal(1,"FAIL logic_gate vector=%0d expected=%0h actual=%0h",n,expected,{x,y,z});
            checked=checked+1;
        end
        if(checked!=4) $fatal(1,"Incomplete test");
        $display("LAB1_PASS logic_gate cases=%0d",checked);
        $finish;
    end
    initial begin #1000000; $fatal(1,"Watchdog"); end
endmodule
```

PASS 로그와 파형

[illegible]

위의 그림을 보면 테스트벤치에서 네 입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다. 모든 케이스가 정상적으로 출력되어 40ns에 종료됨을 볼 수 있다.

네 시간 구간 정리 및 OR과 XOR의 차이

	a	b	x	y	z
0~10 (ns)	0	0	0	0	0
10~20 (ns)	0	1	0	1	1
20~30 (ns)	1	0	0	1	1
30~40 (ns)	1	1	1	1	0

OR은 두 입력 중 하나 이상이 1이면 출력이 1이 된다. XOR은 두 입력이 서로 다를 때 출력이 1이 된다. 따라서 두 입력이 모두 1인 11인 30~40ns일 때 OR은 1, XOR은 0이 된다.

보드에서 확인할 입력과 출력

실제 FPGA 보드에서 스위치 같은 입력을 바꿨을 때 LED 출력이 예상대로 나오는지 확인하면 된다. 즉 입력 a, b를 바꿔보고 출력인 x, y, z의 LED 출력이 어떻게 되는지 확인한다. 따라서 실제로 확인할 조합은 4가지로 출력을 확인한다.

3. 실험 2 전가산기

목적, 포트, 비트 폭, 예상표와 동작 원리

1비트 full adder를 설계하고 시뮬레이션하여 입력 a, b, cin에 따른 합 s와 자리올림 cout이 정상적으로 출력되는지 확인한다.

포트는 a, b, cin이 input이고 s, cout이 output으로 모두 비트 폭은 1비트로 동일하다. 예상표는 다음과 같다.

a	b	cin	s	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

동작원리는 full adder가 half adder를 두 개 연결한 것에서 시작한다. 첫 번째 half adder에서 a와 b를 더하여 partial_sum과 carry_ab를 생성한다. 그 다음 두 번째 half adder에서 partial_sum과 cin을 더하여 s와 carry_cin을 생성한다. 마지막으로 carry_ab와 carry_cin을 OR 게이트를 통과시켜 cout을 생성한다.

소스, TB경로, 코드커밋

-소스: half_adder; https://github.com/thechef030/lab1_02_fulladder/blob/main/src/half_adder.v

full_adder; https://github.com/thechef030/lab1_02_fulladder/blob/main/src/full_adder.v

-TB경로: https://github.com/thechef030/lab1_02_fulladder/blob/main/sim/tb_full_adder.v

-커밋 해쉬: https://github.com/thechef030/lab1_02_fulladder/commit/fc4a417d5ca7fd4e47c352d2bf2c55604008b79

입력 조합, 기댓값, 자동 비교, 종료 조건

a, b, cin을 input으로 하는 입력 조합은 모두 1비트이므로 8가지가 나온다. 그리고 기댓값은 다음의 코드와 같다.

```
expected = int'(a) + int'(b) + int'(cin);
```

a와 b와 cin을 모두 더한 값과 {cout, s}로 만든 2비트 수와 동일하면 시뮬레이션이 계속 진행되고 틀리면 시뮬레이션이 종료된다. 이에 따라서 자동비교가 발생한다. 종료 조건은 이 자동비교가 8개의 입력 조합에 대해서 모두 검사가 이루어지는 것이다. 따라서 8개가 모두 정상 검사되면 종료된다.

```
timescale 1ns/1ps

module tb_full_adder;

    reg a, b, cin;
    wire s, cout;

    full_adder dut(a, b, cin, s, cout);

    integer n, checked = 0;
    reg [1:0] expected;

    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0, tb_full_adder);

        for (n = 0; n < 8; n = n + 1) begin
            (a, b, cin) = n;

            expected = int'(a) + int'(b) + int'(cin);

            #10;

            if ((cout, s) != expected)
                $fatal(1,
                    "FAIL full_adder vector=%0d expected=%0h actual=%0h",
                    n, expected, (cout, s));

            checked = checked + 1;
        end

        if (checked != 8)
            $fatal(1, "Incomplete test");

        $display("LAB1_PASS full_adder cases=%0d", checked);
    end

endmodule
```

```
$finish;
end

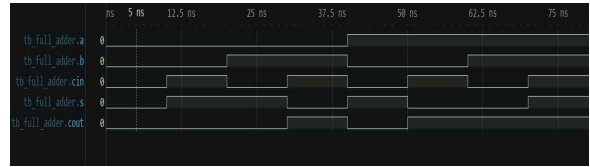
initial begin
    #100000;
    $fatal(1, "Watchdog");
end

endmodule
```

PASS 로그와 파형

```
Executing task in folder PROJECT: C:\Users\Tjgus\AppData\Local\Programs\Python\Python311\python.exe tools/lab1.py simulate
warning: Some design elements have no explicit time unit and/or
: time precision. This may cause confusing timing results.
: Affected design elements are:
:   -- module full_adder declared here: C:\Users\Tjgus\lab1_02_full_adder\src\full_adder.v:1
:   -- module half_adder declared here: C:\Users\Tjgus\lab1_02_full_adder\src\half_adder.v:1
Run C:\msys64\mingw64\bin\vvp.exe -K C:\Users\Tjgus\lab1_02_full_adder\build\sim-run-38e5447d63472ba8f5e77dacc097\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB1_PASS full_adder cases=8
C:\Users\Tjgus\lab1_02_full_adder\src\tb_full_adder.sv:37: $finish called at 80000 (tps)
시뮬레이션 실행 완료. 전리표 자기검사-파형으로 회로의 정상 동작 여부를 확인하세요.
실행: C:\Users\Tjgus\lab1_02_full_adder\build\sim-run-38e5447d63472ba8f5e77dacc097\sim.vvp
Terminal will be reused by tasks, press any key to close it.
```

위의 그림을 보면 테스트벤치에서 네입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다. 모든 케이스가 정상적으로 출력되어 80ns에 종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

다음의 표는 시간 구간별 값 해석이다.

시간(ns)	a	b	cin	s	cout
0~10	0	0	0	0	0
10~20	0	0	1	1	0
20~30	0	1	0	1	0
30~40	0	1	1	0	1
40~50	1	0	0	1	0
50~60	1	0	1	0	1
60~70	1	1	0	0	1
70~80	1	1	1	1	1

다음의 표는 파형의 예상값이다.

시간(ns)	a	b	cin	s	cout
5	0	0	0	0	0
15	0	0	1	1	0
35	0	1	1	0	1
45	1	0	0	1	0
75	1	1	1	1	1

vaporview에서 구한 파형의 결과값과 동일함을 확인할 수 있다.

보드에서 확인할 입력과 출력

실제 FPGA 보드에서 스위치 같은 입력을 바꿨을 때 LED 출력이 예상대로 나오는지 확인하면 된다. 즉 입력 a, b, cin을 바꿔보고 출력인 s, cout이 LED 출력이 어떻게 되는지 확인한다. 따라서 실제로 확인할 조합은 8가지로 출력을 확인한다.

4. 실험 3 4비트 가산기

목적, 포트, 비트 폭, 예상표와 동작 원리

4비트의 두 입력에 대해 가산기를 작동시킨다. a와 b는 4비트 입력이고 출력으로 하위 4비트 s, 최상위 1비트를 cout으로 한다. 따라서 a와 b 4비트 입력에 대하여 5비트 출력을 커버할 수 있도록 s와 cout을 둔다. 따라서 예상표는 다음과 같다.

a	b	s	cout
0000	0000	0000	0
0000	0001	0001	0
0000	0011	0011	0
1000	0000	1000	0
1111	1111	1110	1

동작 원리로는 a가 0일 때 b를 0~15를 입력하고 a가 1일 때 b를 또 다시 0~15를 인가하여 모든 경우의 수를 만들어낸다. a와 b가 모두 1000인 경우 10000이 출력된다. 따라서 cout이 1이 되고 s는 0000이 되는 원리로 작동한다.

소스, TB경로, 코드커밋

-소스: https://github.com/thechef030/lab1_03_adder4/blob/main/src/adder_4bit.v

-TB경로: https://github.com/thechef030/lab1_03_adder4/blob/main/sim/tb_adder_4bit.v

-커밋 해쉬: https://github.com/thechef030/lab1_03_adder4/commit/f5a05bd609eb34eca4fccd6d90d7959c6ef646cd

입력 조합, 기댓값, 자동 비교, 종료 조건

앞서 설명했듯이 a와 b는 모두 4비트이므로 16*16의 경우의 수가 존재하므로 총 256의 입력 조합이 발생한다. 교안에서 제시한 구간 중간값을 예시로 들면 기댓값은 다음과 같다.

시간(ns)	a	b	s	cout
5	0000	0000	0000	0
15	0000	0001	0001	0
35	0000	0011	0011	0
1285	1000	0000	1000	0
2555	1111	1111	1110	1

$n/16$ 은 a, $n\%16$ 은 b다. 16x16개 조합의 합을 5비트 기대값에 담아 carry까지 검사한다. {cout, s}로 만든 5비트 수와 동일하면 시뮬레이션이 계속 진행되고 틀리면 시뮬레이션이 종료된다. 이에 따라서 자동비교가 발생한다. 종료 조건은 이 자동비교가 256개의 입력 조합에 대해서 모두 검사가 이루어지는 것이다. 따라서 256개가 모두 정상 검사되면 종료된다.

```
timescale 1ns/1ps
module tb_adder_4bit;
    reg [3:0] a,b; wire [3:0] s; wire cout; adder_4bit dut(a,b,s,cout);
    integer n,checked=0;
    reg [4:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_adder_4bit);

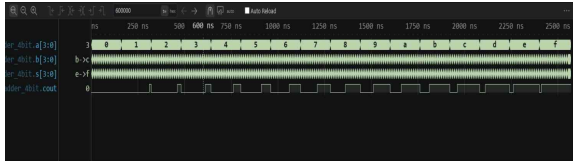
        for(n=0;n<256;n=n+1) begin
            {a,b}=n;
            expected=(n/16)*(n%16);
            #10;
            if ((cout,s) != expected)
                $fatal(1,"FAIL adder_4bit vector=%0d expected=%h actual=%h",n,expected,(cout,s));
            checked=checked+1;
        end
        if(checked!=256) $fatal(1,"Incomplete test");
        $display("LAB1_PASS adder_4bit cases=%0d",checked);
        $finish;
    end
    initial begin #1000000; $fatal(1,"Watchdog"); end
endmodule
```

PASS 로그와 파형



위의 그림을 보면 테스트벤치에서 두입력에 대해 검증을 수행한 결과 모든 케이스가 정

상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다. 모든 케이스가 정상적으로 출력되어 2560ns에 종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

시간 구간별로 모든 값을 살펴보려면 256가지의 경우를 모두 생각해야 한다. 이는 비효율적이므로 랜덤으로 구간을 뽑아서 해석해보면 310~320ns를 해석해보자. 이 때 a는 1이고 b는 15이다. 따라서 합은 16이 되고 이를 2진수로 표현하면 10000이므로 cout이 1이 되고 s가 0000이 됨을 알 수 있다. 앞서 다른 교안에 제시된 구간 중간값을 예시로 기댓값과 실제 파형을 비교하면 동일함을 확인할 수 있다.

보드에서 확인할 입력과 출력

DIP1~4가 a, DIP5~8이 b, LED1이 cout, LED2-5가 s이다. 예를 들어서 a = 1111, b = 1111을 넣었을 때 LED가 cout = 1, s = 1110으로 나오는지 확인하면 된다.

5. 실험 4 4비트 감산기

목적, 포트, 비트 폭, 예상표와 동작 원리

4비트의 두 입력에 대해 가산기를 작동시킨다. a와 b는 4비트 입력이고 출력으로 하위 4비트는 d, bor은 $a < b$ 일 때만 1으로 한다. 같은 수끼리 빼면 borrow는 0이다. 이에 따라 예상표는 다음과 같다.

a	b	d	bor
0000	0000	0000	0
0000	0001	1111	1
1111	0011	1111	0

동작 원리로는 a가 0일 때 b를 0~15를

입력하고 a가 1일 때 b를 또 다시 0~15를 인가하여 모든 경우의 수를 만들어낸다. 예를 들어 a가 0011이고 b가 0101일 경우 0011-0101로 borrow가 1이 되고 1110이 d로 출력된다. 따라서 bor이 1이 되고 d는 1110이 되는 원리로 작동한다.

소스, TB경로, 코드커밋

-소스: https://github.com/thechef030/lab1_04_subtractor4/blob/main/src/sub_4bit.v

-TB경로: https://github.com/thechef030/lab1_04_subtractor4/blob/main/sim/tb_sub_4bit.sv

-커밋 해쉬: https://github.com/thechef030/lab1_04_subtractor4/commit/c0df4501bafec27225904136c8c16692b606c647

입력 조합, 기댓값, 자동 비교, 종료 조건

앞서 설명했듯이 a와 b는 모두 4비트이므로 16*16의 경우의 수가 존재하므로 총 256의 입력 조합이 발생한다. 256개 입력 조합을 모두 검사한다. 각 자극은 10ns, 종료 시각은 2560ns다. 교안에서 제시한 구간 중간값을 예시로 들면 기댓값은 다음과 같다.

시간(ns)	a	b	d	bor
5	0000	0000	0000	0
15	0000	0001	1111	1
35	0000	0011	1101	1
1285	1000	0000	1000	0
2555	1111	1111	0000	0

$n/16$ 은 a, $n\%16$ 은 b다. a가 b보다 크거나 같으면 0, 작으면 16이 된다. 또한 a-b에 대해서 15와 AND 게이트를 통과시켜 하위 4비트만 남긴다. 이렇게 5비트의 expected 값을 만들어내고 {bor, d}로 만든 5비트 수와 동일하면 시뮬레이션이 계속 진행되고 틀리면 시뮬레이션이 종료된다. 이에 따라서 자동비교가 발생한다. 종료 조건은 이 자동비교가 256개의 입력 조합에 대해서 모두 검사가 이루어지는 것이다. 따라서 25

6개가 모두 정상 검사되면 종료된다.

```

timescale 1ns/1ps

module tb_sub_4bit;

    reg [3:0] a,b; wire [3:0] d; wire bor; sub_4bit dut(a,b,d,bor);
    integer n,checked=0;
    reg [4:0] expected;

    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_sub_4bit);

        for(n=0;n<256;n=n+1) begin
            (a,b)=n;
            expected((((n/16)<(n%16)) > 16 : 0) | (((n/16)-(n%16)) & 15);
            #10;
            if ((bor,d) != expected)
                $fatal(1,"FAIL sub_4bit vector=%0d expected=%h actual=%h",n,expected,(bor,d));
            checked=checked+1;
        end

        if(checked!=256) $fatal(1,"Incomplete test");
        $display("LAB1_PASS sub_4bit cases=%0d",checked);
        $finish;
    end

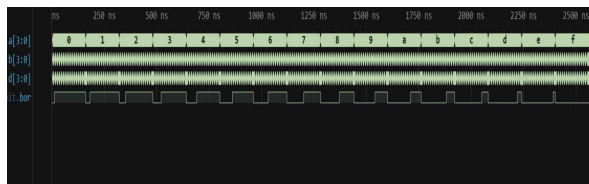
    initial begin #100000; $fatal(1,"Watchdog"); end
endmodule

```

PASS 로그와 파형

[illegible]

위의 그림을 보면 테스트벤치에서 두입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다. 모든 케이스가 정상적으로 출력되어 2560ns에 종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

시간 구간별로 모든 값을 살펴보려면 256가지의 경우를 모두 생각해야 한다. 이는 비효율적이므로 랜덤으로 구간을 뽑아서 해석해보면 10~20ns와 170~180ns를 해석해보자. 10~20ns일 때 a는 0이고 b는 1이다. 따라서 bor는 1이 되고 d는 1111이 된다. 그 다음으로 170~180ns일 때 a는 1이고 b는 1이다. 따라서 차는 0이 되고 이를 2진수로 표현하면 0000이므로 bor이 0이 되고 d가 0000이 됨을 알 수 있다. 앞서 다루고

안에 제시된 구간 중간값을 예시로 기댓값과 실제 파형을 비교하면 동일함을 확인할 수 있다.

보드에서 확인할 입력과 출력

DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=bor, LED2-5=d[3:0]이다. 예를 들어서 a = 1111, b = 1111을 넣었을 때 LED가 bor = 0, d = 0000으로 나오는지를 확인하면 된다.

6. 실험 5 크기 비교기

목적, 포트, 비트 폭, 예상표와 동작 원리

4비트의 두 입력에 대해서 크기를 비교하는 회로를 설계한다. a 와 b 는 4비트 입력이고 o 를 3비트 출력으로 할당한다. 동작 원리로는, $o[2]$ 는 $a > b$, $o[1]$ 은 $a = b$, $o[0]$ 은 $a < b$ 를 나타낸다. 세 결과 중 하나만 1이다. 이에 따라서 실행 전에 교안에 있는 조건에 따라 예상값을 작성하면 다음과 같다.

a	b	o
0000	0000	010
0000	0001	001
0001	0000	100

소스, TB경로, 코드커밋

-소스: https://github.com/thechef030/lab1_05_comparator4/blob/main/src/compare_4.v
-TB경로: https://github.com/thechef030/lab1_05_comparator4/blob/main/sim/tb_comp_are_4.sv
-커밋 해쉬: https://github.com/thechef030/lab1_05_comparator4/commit/47565ebb0a31c619065ff862ce7fe125a9568fc2

입력 조합, 기댓값, 자동 비교, 종료 조건

앞서 설명했듯이 a와 b는 모두 4비트이므로 16*16의 경우의 수가 존재한다. 따라서 총 256의 입력 조합이 발생한다. 256개 입

력 조합을 모두 검사한다. 각 자극은 10ns, 종료 시각은 2560ns다. 교안에서 제시한 구간 중간값을 예시로 들면 기댓값은 다음과 같다.

시간(ns)	a	b	o
5	0000	0000	010
15	0000	0001	001
35	0000	0011	001
1285	1000	0000	100
2555	1111	1111	010

테스트벤치의 코드를 보면 $n/16$ 은 a, $n\%16$ 은 b다. a가 b보다 클 때 4를 출력하고 이것이 거짓일 때 a와 b가 같은지 판별한다. 이것이 참이라면 2를 출력하고 거짓이면 1을 출력한다. 이렇게 expected 값을 생성한다. 4, 2, 1을 각각 3비트 2진수로 표현하면 100, 010, 001이 됨을 쉽게 알 수 있다. 따라서 이 값과 o값이 같은지 자동 비교하여 256가지의 경우의 수를 모두 검증한다. 종료 조건은 이 자동비교가 256개의 입력 조합에 대해서 모두 검사가 이루어지는 것이다. 따라서 256개가 모두 정상 검사되면 종료된다.

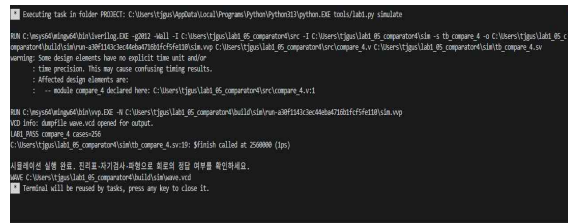
```

timescale 1ns/1ps
module tb_compare_4;
    reg [3:0] a,b; wire [2:0] o; compare_4 dut(a,b,o);
    integer n,checked=0;
    reg [2:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_compare_4);

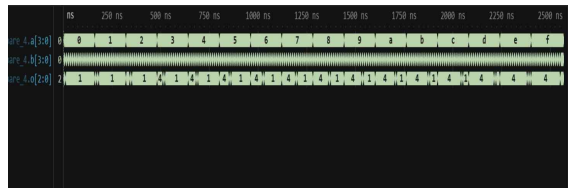
        for(n=0;n<256;n=n+1) begin
            (a,b)=n;
            expected=((n/16)>(n%16)) ? 4 : (((n/16)==(n%16)) ? 2 : 1);
            #10;
            if (o != expected)
                $fatal(1,"FAIL compare_4 vector=%0d expected=%0d actual=%0d",n,expected,o);
            checked=checked+1;
        end
        if(checked!=256) $fatal(1,"Incomplete test");
        $display("LAB1_PASS compare_4 cases=%0d",checked);
        $finish;
    end
    initial begin #100000; $fatal(1,"Watchdog"); end
endmodule

```

PASS 로그와 파형



위의 그림을 보면 테스트벤치에서 두 입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다. 모든 케이스가 정상적으로 출력되어 2560ns에 종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

시간(ns)	a	b	o
0~10	0000	0000	010
10~20	0000	0001	001
160~170	0001	0000	100

256가지의 시간 구간별 값을 모두 알아볼 수 없으므로 위의 표는 실행 전에 예상했던 시간 구간을 vaporview에서 확인한 값으로 재작성하였다. 위의 표와 실행 전에 예상값을 비교하면 동일함을 확인할 수 있다.

보드에서 확인할 입력과 출력

DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=큼, LED2=같음, LED3=작음으로 o의 출력에 따라서 LED의 숫자에 맞게 켜지는지 확인한다.

7. 실험 6 4:1 멀티플렉서

목적, 포트, 비트 폭, 예상표와 동작 원리

입력으로는 4비트 입력인 i, 2비트 입력인 s가 있고 출력으로는 z가 있다. 선택 s=00,

01, 10, 11은 각각 $i[3]$, $i[2]$, $i[1]$, $i[0]$ 을 z 로 보낸다. 원본의 비트 순서를 따른다. 이에 따라서 실행 전에 교안에 있는 조건에 따라 예상값을 작성하면 다음과 같다.

I	s	z
1000	00	1
1000	01	0
0001	11	1

동작 원리를 살펴보면 4:1 멀티플렉서는 4비트 입력에 대하여 1비트를 선택하여 출력하는 회로이다. 또한 원본의 비트 순서를 따르므로 s 의 값이 00일 때 i 의 3번째에 해당하는 1비트 정보를 출력하게 된다.

소스, TB경로, 코드커밋

-소스: https://github.com/thechef030/lab1_06_mux4to1/blob/main/src/mux_4x1.v

-TB경로: https://github.com/thechef030/lab1_06_mux4to1/blob/main/sim/tb_mux_4x1.sv

-커밋 해쉬: https://github.com/thechef030/lab1_06_mux4to1/commit/38d72cfb8d0f1f3e71618b08a165672a3bf28e70

입력 조합, 기댓값, 자동 비교, 종료 조건

4비트 정보에 대해서 16개의 경우의 수가 발생하고 16개의 경우의 수에 대하여 각각 4개의 선택이 발생하므로 64개의 입력 조합이 발생한다. 64개 입력 조합을 모두 검사한다. 각 자극은 10ns, 종료 시각은 640ns다. 교안에서 제시한 구간 중간값을 예시로 들면 기댓값은 다음과 같다.

시간(ns)	I	s	z
5	0000	00	0
15	0000	01	0
35	0000	11	0
325	1000	00	1
635	1111	11	1

$n=0\sim63$ 에 대해 $\{i,s\}=n$ 이므로 $i=n/4$, $s=n\%4$ 다. 기댓값은 $((n/4) \gg (3-(n\%4))) \& 1$ 로 계산한다. 자동 비교하여 64가지의 경우의 수를 모두 검증한다. 종료 조건은 이 자동비교가 64개의 입력 조합에 대해서 모두 검사가 이루어지는 것이다. 따라서 각 입력에 대해 10ns 유지하므로 640ns에 정상종료된다.

```

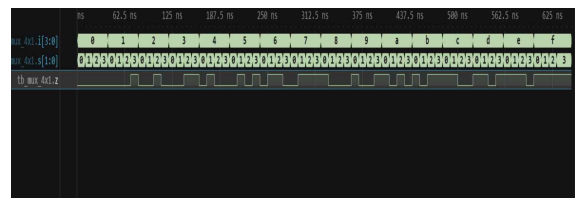
`timescale 1ns/1ps
module tb_mux_4x1;
    reg [3:0] i; reg [1:0] s; wire z; mux_4x1 dut(i,s,z);
    integer n,checked=0;
    reg [0:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_mux_4x1);

        for(n=0;n<64;n=n+1) begin
            {i,s}=n;
            expected=((n/4) >> (3-(n%4))) & 1;
            #10;
            if (z != expected)
                $fatal(1,"FAIL mux_4x1 vector=%0d expected=%0h actual=%0h",n,expected,z);
            checked=checked+1;
        end
        if(checked!=64) $fatal(1,"Incomplete test");
        $display("LAB1_PASS mux_4x1 cases=%0d",checked);
        $finish;
    end
    initial begin #100000; $fatal(1,"Watchdog"); end
endmodule

```

PASS 로그와 파형

위의 그림을 보면 테스트벤치에서 두 입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다. 모든 케이스가 정상적으로 출력되어 640ns에 종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

시간(ns)	I	s	z
--------	---	---	---

320~330	1000	00	1
330~340	1000	01	0
340~350	1000	10	0
350~360	1000	11	0

320~360ns에는 $i=1000$ 이 유지되며 s 만 변한다. 처음 10ns에는 $i[3]=1$ 을 선택하고, 나머지 구간에서는 0인 비트를 선택하므로 z 는 1, 0, 0, 0 순서다. 예상값과 vscode로 실행한 vaporview의 값과 동일함을 확인할 수 있다.

보드에서 확인할 입력과 출력

KEY1-4= $i[3:0]$, DIP1-2= $s[1:0]$. LED1= z 이다. $i=1000$ 에서 $s=00$ 일 때만 LED1이 켜지는지 확인한다. $i=0001$ 에서는 $s=11$ 일 때 켜져야 한다.

8. 실험 7 1:8 디멀티플렉서

목적, 포트, 비트 폭, 예상표와 동작 원리

하나의 입력을 선택선이 지정한 출력으로 전달하는 조합회로를 설계하고 검증한다. 입력 i 는 1비트, s 은 3비트이고 출력 o 은 8비트다. 선택된 o 에 i 를 전달하며, 나머지 출력은 0이다. $i=0$ 이면 선택선과 관계없이 모든 출력이 0이 된다. 예상값은 다음과 같다.

I	S	O
0	모든 값	00000000
1	000	10000000
1	001	01000000
1	010	00100000
1	011	00010000
1	100	00001000
1	101	00000100
1	110	00000010
1	111	00000001

소스, TB경로, 코드커밋

-소스: https://github.com/thechef030/lab1_07_demux1to8/blob/main/src/demux_1x8.v

-TB경로: https://github.com/thechef030/lab1_07_demux1to8/blob/main/sim/tb_demux_1x8.sv

-커밋 해쉬: https://github.com/thechef030/lab1_07_demux1to8/commit/ac01e728f901e53004d5d3e0e6ce77f0fe7bd9ef

입력 조합, 기댓값, 자동 비교, 종료 조건

i 의 2가지와 s 의 8가지를 조합하여 총 16개의 입력 조합이 발생한다. 각 입력은 10ns 유지하며 정상 종료는 160ns이다.

테스트벤치의 코드를 보면 $i=n/8$, $s=n\%8$ 이다. n 이 8보다 작을 때 i 는 0이므로 expected는 0이 된다. n 이 8보다 크거나 같을 때, i 는 1이 되므로 s 에 따라서 한 비트만 1이 되도록 expected를 만들었다. 따라서 이렇게 생성된 expected값과 실제 출력인 o 의 값을 자동비교하여 시뮬레이션한다. 종료 조건은 이 자동비교가 16개의 입력 조합에 대해서 모두 검사가 이루어지는 것이다. 따라서 16개가 모두 정상 검사되면 종료된다.

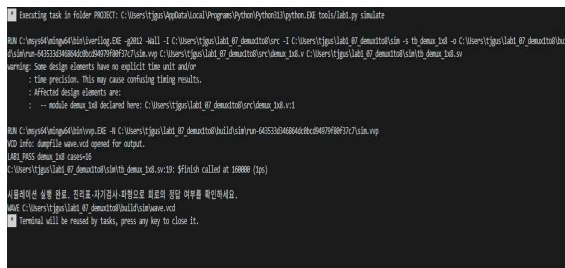
```

`timescale 1ns/1ps
module tb_demux_1x8;
    reg i; reg [2:0] s; wire [7:0] o; demux_1x8 dut(i,s,o);
    integer n,checked=0;
    reg [7:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_demux_1x8);

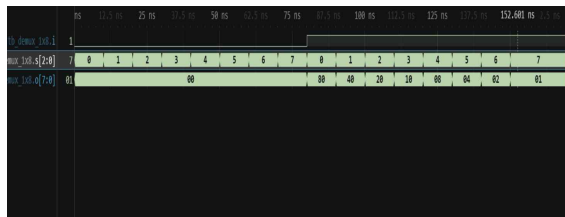
        for(n=0;n<16;n=n+1) begin
            {i,s}=n;
            expected=(n>=8) ? (128 >> (n%8)) : 0;
            #10;
            if (o != expected)
                $fatal(1,"FAIL demux_1x8 vector=%0d expected=%h actual=%h",n,expected,o);
            checked=checked+1;
        end
        if(checked!=16) $fatal(1,"Incomplete test");
        $display("LAB1_PASS demux_1x8 cases=%0d",checked);
        $finish;
    end
    initial begin #100000; $fatal(1,"Watchdog"); end
endmodule

```

PASS 로그와 파형



위의 그림을 보면 테스트벤치에서 두 입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다. 모든 케이스가 정상적으로 출력되어 160ns에 종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

시간(ns)	I	s	o
0~10	0	000	00000000
70~80	0	111	00000000
80~90	1	000	10000000
90~100	1	001	01000000
150~160	1	111	00000001

0~80ns에는 $i=0$ 이므로 모든 출력이 0이다. 80~90ns에는 $i=1$, $s=000$ 이므로 $o[7]$ 만 1이다. 선택선이 증가할 때마다 1의 위치가 오른쪽으로 이동하며, 150~160ns에는 $o[0]$ 만 1이 된다. 교안의 85ns와 155ns 값도 각각 10000000과 00000001이므로 계산한 예상값과 일치한다.

보드에서 확인할 입력과 출력

KEY1 = i, DIP1~3 = s, LED1~8 = o이다. i=0에서 모든 LED가 꺼지는지 확인한다. i=1에서 s=000이면 LED1, s=111이면 LED8이 켜져야 한다. s를 000부터 111까지 바꾸며

점등 위치가 LED1에서 LED8로 이동하는지
확인한다.

9. 실험 8 8:3 인코더

목적, 포트, 비트 폭, 예상표와 동작 원리

8비트 입력에서 1인 비트의 위치를 3비트 번호로 변환하는 회로를 설계한다. 입력 i 은 8비트, 출력 a 은 3비트다. $i[7]$ 만 1이면 000, $i[0]$ 만 1이면 111을 출력한다. 0 또는 다중 입력은 000을 출력한다. 예상표는 다음과 같다.

I	a
10000000	000
01000000	001
00100000	010
00010000	011
00001000	100
00000100	101
00000010	110
00000001	111
00000011	000(잘못된 입력)

소스, TB경로, 코드커밋

-소스: https://github.com/thechef030/lab1_08_encoder8to3/blob/main/src/encoder8x3.v

-TB경로: https://github.com/thechef030/lab_1_08_encoder8to3/blob/main/sim/tb_encoder8x3.sv

-커밋 해쉬: https://github.com/thechef030/lab1_08_encoder8to3/commit/ff372aca582720837d697e3870cab5b97a2bd7e0

입력 조합, 기댓값, 자동 비교, 종료 조건

1은 8비트이므로 256의 입력 조합이 발생한다. 8개 one-hot 입력뿐 아니라 256개 입력을 모두 넣는다. 1이 없거나 여러 개인 입력의 기대값은 0.00이다.

테스트벤치의 코드를 보면 처음에 expecte

d값을 000으로 초기화해둔다. $l = n$ 이고 k 라는 수를 expected값으로 설정한다. 128>>k로 k를 이진수로 표현하고 이를 n과 비교한다. 이 때 n값과 0부터 7의 값을 갖는 k와 같을 때 3비트 이진수를 expected값으로 출력한다. 이와 a의 값을 자동비교하여 시뮬레이션한다. 종료 조건은 이 자동비교가 256개의 입력 조합에 대해서 모두 검사가 이루어지는 것이다. 따라서 256개가 모두 정상 검사되면 종료된다. 각 자극은 10ns 유지되므로 종료 시각은 2560ns이다.

```

`timescale 1ns/1ps

module tb_encoder8x3;
    reg [7:0] i; wire [2:0] a; integer k; encoder8x3 dut(i,a);
    integer n,checked=0;
    reg [2:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_encoder8x3);

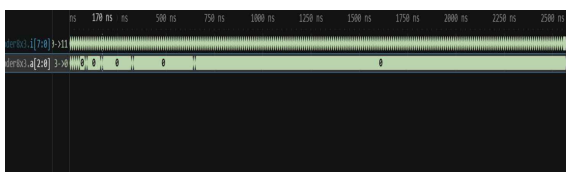
        for(n=0;n<256;n=n+1) begin
            i=n;
            expected=0;
            for(k=0;k<8;k=k+1) if(n==(128>k)) expected=k;
            #10;
            if (a != expected)
                $fatal(1,"FAIL encoder8x3 vector=%0d expected=%h actual=%h",n,expected,a);
            checked=checked+1;
        end
        if(checked!=256) $fatal(1,"Incomplete test");
        $display("LAB1_PASS encoder8x3 cases=%0d",checked);
        $finish;
    end
    initial begin #100000; $fatal(1,"Watchdog"); end
endmodule

```

PASS 로그와 파형

[illegible]

위의 그림을 보면 테스트벤치에서 하나의 입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다.
모든 케이스가 정상적으로 출력되어 2560n

s에 종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

시간(ns)	I	a
0~10	00000000	000
10~20	00000001	111
20~30	00000010	110
30~40	00000011	000
1280~1290	10000000	000
2550~2560	11111111	000

10~20ns에는 i[0]만 1이므로 번호 7에 해당하는 111을 출력한다. 30~40ns에는 두 비트가 동시에 1이므로 유효한 one-hot 입력이 아니며 000을 출력한다. 1280~1290ns에는 i[7]만 1이므로 유효한 부호 000이 나온다. 15, 35, 1285ns에서 계산한 값은 교안의 값과 일치한다.

보드에서 확인할 입력과 출력

KEY1~8 = i, LED1~3 = a이다. KEY8만 논리 1로 설정하면 111이 출력되어 LED1~3이 모두 켜져야 한다. KEY7만 1이면 110이므로 LED1과 LED2가 켜진다. KEY1만 1인 입력, 모든 입력이 0인 경우, 다중 입력에서 모두 000이 나타나는지도 비교한다.

10. 실험 9 3:8 디코더

목적, 포트, 비트 폭, 예상표와 동작 원리

3비트 입력에 대응하는 출력 하나만 1로 만드는 디코더를 설계하고 검증한다. 각각 1비트 입력 a , b , c 에 대하여 7비트 출력 o 를 발생시킨다. a 가 최상위 비트, c 가 최하위 비트이다. 예상표는 다음과 같다.

a	b	c	o
0	0	0	00000001
0	0	1	00000010

0	1	0	00000100
0	1	1	00001000
1	0	0	00010000
1	0	1	00100000
1	1	0	01000000
1	1	1	10000000

소스, TB경로, 코드커밋

-소스: https://github.com/thechef030/lab1_09_decoder3to8/blob/main/src/decoder3x8.v

-TB경로: https://github.com/thechef030/lab_1_09_decoder3to8/blob/main/sim/tb_decoder3x8.sv

-커밋 해쉬: https://github.com/thechef030/lab1_09_decoder3to8/commit/3d0665abaa414228f0e71a5bf762c6976b6539cd

입력 조합, 기댓값, 자동 비교, 종료 조건

각각 1비트인 세 입력에 대하여 입력 조합은 2^3 으로 8개이다. 테스트벤치의 코드를 보면 $n=0\sim7$ 에 대해 $\{a,b,c\}=n$ 을 인가하고 기댓값을 $1 < n$ 으로 계산한다. 따라서 abc의 값에 따라서 8비트정보에서의 1의 위치가 정해진다. 이를 o와 자동비교하여 8가지 시뮬레이션을 시행한다. 각 입력은 10ns 유지하며 8개 검사가 모두 통과하면 80ns에 종료한다.

```
timescale 1ns/1ps
module tb_decoder3x8;
    reg a,b,c; wire [7:0] o; decoder3x8 dut(a,b,c,o);
    integer n,checked=0;
    reg [7:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0,tb_decoder3x8);

        for(n=0;n<8;n=n+1) begin
            (a,b,c)=n;
            expected=1 << n;
            #10;
            if (o != expected)
                $fatal(1,"FAIL decoder3x8 vector=%0d expected=%h actual=%h",n,expected,o);
            checked=checked+1;
        end
        if(checked!=8) $fatal(1,"Incomplete test");
        $display("LAB1_PASS decoder3x8 cases=%0d",checked);
        $finish;
    end
    initial begin #1000000; $fatal(1,"Watchdog"); end
endmodule
```

PASS 로그와 파형

```

$ docker task --help PROXY: C:\Users\jgaladi\AppData\Local\Programs\Python\Python38\python.exe task\cli.py simulate

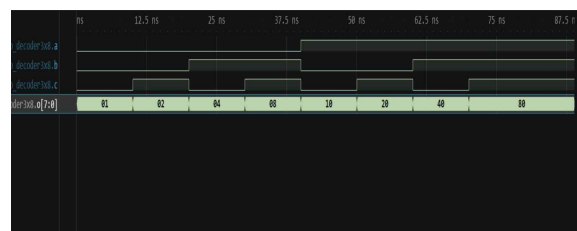
RUN C:\msys64\mingw64\bin\perl.exe -dperl -dperl -I C:\Users\jgaladi_0_decoder\src\perl -I C:\Users\jgaladi_0_decoder\src\perl -s -t decoder_0 -C C:\Users\jgaladi_0_decoder\src\perl\src\perl -DPERL5LIB=C:\Users\jgaladi_0_decoder\src\perl\src\perl\lib\perl5\site_perl\bin C:\Users\jgaladi_0_decoder\src\perl\src\perl\bin\perl.exe
warning: Some design elements have no explicit time unit and
: time precision. This may cause confusing timing results.
:
: Affected design elements are:
:
: - module decoder_0 declared here: C:\Users\jgaladi_0_decoder\src\perl\src\perl.vci

RUN C:\msys64\mingw64\bin\perl.exe -dC:\Users\jgaladi_0_decoder\src\perl\src\perl -DPERL5LIB=C:\Users\jgaladi_0_decoder\src\perl\src\perl\lib\perl5\site_perl\bin C:\Users\jgaladi_0_decoder\src\perl\src\perl\bin\perl.exe
info: default vnc-rcd opened for output.
INFO: PERL decoder_0 uses:
C:\Users\jgaladi_0_decoder\src\perl\src\perl.vci: Vncfile called at: 0000 (ip)

이제와서 보면 설명대로, 진리값은 20가지지만, 최종으로 필요한 값은 4개만 확인해볼게요.
$ docker C:\Users\jgaladi_0_decoder\src\perl\src\perl.vci: Vncfile called at: 0000 (ip)
$ perl
perl will be used by tasks, press any key to close it.

```

위의 그림을 보면 테스트벤치에서 세 입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다.
모든 케이스가 정상적으로 출력되어 80ns에
종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

구간(ns)	abc	o
0~10	000	00000001
10~20	001	00000010
20~30	010	00000100
30~40	011	00001000
40~50	100	00010000
50~60	101	00100000
60~70	110	01000000
70~80	111	10000000

0~10ns에는 입력이 000이므로 출력은 00000001이다. 입력 번호가 증가하면 1의 위치가 왼쪽으로 이동한다. 30~40ns에는 00001000, 70~80ns에는 10000000이 된다. 교안의 5, 35, 75ns 값과 계산한 예상값이 각각 일치한다.

보드에서 확인할 입력과 출력

KEY1 = a, KEY2 = b, KEY3 = c0이며

LED1~8 = o이다. abc=000이면 o[0]에 연결된 LED8이 켜지고, abc=011이면 LED5가 켜진다. abc=111이면 o[7]에 연결된 LED1이 켜지는지 확인한다. 모든 입력에서 LED가 하나씩만 켜져야 한다.

11. 실험 10 7세그먼트 디코더

목적, 포트, 비트 폭, 예상표와 동작 원리

4비트 입력을 7세그먼트 표시 패턴으로 변환하는 회로를 설계한다. 입력 bcd은 4비트, 출력 seg_data은 8비트다. 4비트 입력의 0~F를 표시한다. 출력 비트 순서는 a, b, c, d, e, f, g, dp이며 1일 때 점등한다. dp는 항상 0이다. 이름은 bcd이지만 10~15도 A, b, C, d, E, F로 표시한다.

입력	표시	seg_data	16진 출력
0	0	11111100	FC
1	1	01100000	60
2	2	11011010	DA
3	3	11110010	F2
4	4	01100110	66
5	5	10110110	B6
6	6	10111110	BE
7	7	11100000	E0
8	8	11111110	FE
9	9	11110110	F6
A	A	11101110	EE
B	b	00111110	3E
C	C	10011100	9C
D	d	01111010	7A
E	E	10011110	9E
F	F	10001110	8E

예를 들어 0은 a~f가 켜지고 g와 dp는 꺼져 11111100(FC)이다. 8은 a~g가 모두 켜져 11111110(FE)이며, F는 a, e, f, g가 켜져 10001110(8E)이다.

소스, TB경로, 코드커밋

-소스: https://github.com/thechef030/lab1_10_seven_segment/blob/main/src/seg_deco

der.v

-TB경로: https://github.com/thechef030/lab1_10_seven_segment/blob/main/sim/tb_seg_decoder.v

-커밋 해쉬: https://github.com/thechef030/lab1_10_seven_segment/commit/a1234d9772cbf743c42b983cb18bc7b2b3b8e0a9

입력 조합, 기댓값, 자동 비교, 종료 조건

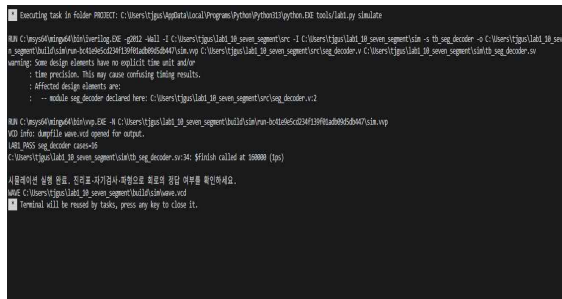
4비트 입력의 16가지 값인 0~F를 차례로 인가한다. 따라서 입력 조합은 16가지가 나온다. digits 배열에는 앞에서 보여준 예상표의 8비트 점등 패턴을 저장하고, 입력 n의 기댓값은 digits[n]으로 정한다. 다음의 코드는 각 입력에 따른 올바른 출력값이다.

```
digits[0]=8'hfc;
digits[1]=8'h60;
digits[2]=8'hda;
digits[3]=8'hf2;
digits[4]=8'h66;
digits[5]=8'hb6;
digits[6]=8'hbe;
digits[7]=8'he0;
digits[8]=8'hfe;
digits[9]=8'hf6;
digits[10]=8'hee;
digits[11]=8'h3e;
digits[12]=8'h9c;
digits[13]=8'h7a;
digits[14]=8'h9e;
digits[15]=8'h8e;
```

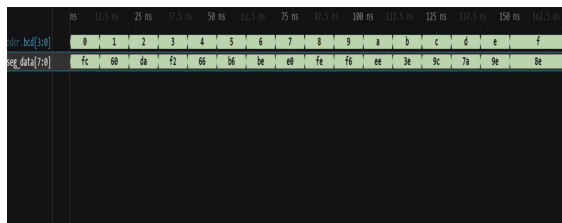
입력이 들어오면 expected값을 가져오고 실제 출력인 seg_data와 비교하여 같으면 통과시킨다. 각 입력을 10ns 유지한 뒤 seg_data와 비교하므로 정상 종료는 160ns다.

```
`timescale 1ns/1ps
module tb_seg_decoder;
    reg [3:0] bcd; wire [7:0] seg_data; reg [7:0] digits[0:15]; seg_decoder dut(bcd, seg_data);
    integer n, checked=0;
    reg [7:0] expected;
    initial begin
        $dumpfile("wave.vcd"); $dumpvars(0, tb_seg_decoder);
        digits[0]=8'hfc;
        digits[1]=8'h60;
        digits[2]=8'hda;
        digits[3]=8'hf2;
        digits[4]=8'h66;
        digits[5]=8'hb6;
        digits[6]=8'hbe;
        digits[7]=8'he0;
        digits[8]=8'hfe;
        digits[9]=8'hf6;
        digits[10]=8'hee;
        digits[11]=8'h3e;
        digits[12]=8'h9c;
        digits[13]=8'h7a;
        digits[14]=8'h9e;
        digits[15]=8'h8e;
        for(n=0; n<16; n++) begin
            bcd=n;
            expected=digits[n];
            #10;
            if(seg_data != expected)
                $fatal(1, "FAIL seg_decoder vector=8'h actual=8'h", n, expected, seg_data);
            checked=checked+1;
        end
        if(checked!=16) $fatal(1, "Incomplete test");
        $display("LAB1_PASS seg_decoder cases=8'h", checked);
        $finish;
    end
    initial begin #1000000; $fatal(1, "Matchdog"); end
endmodule
```

PASS 로그와 파형



위의 그림을 보면 테스트벤치에서 하나의 입력에 대해 검증을 수행한 결과 모든 케이스가 정상적으로 출력되어 통과됨을 알 수 있다.



위의 그림은 vaporview로 실행한 파형이다. 모든 케이스가 정상적으로 출력되어 160ns에 종료됨을 볼 수 있다.

시간 구간별 값 해석 및 예상값과 비교

구간(ns)	bcd	예상 seg_data
0~10	0000	11111100
10~20	0001	01100000
20~30	0011	11110010
80~90	1000	11111110
150~160	1111	10001110

80~90ns에는 입력이 8이므로 a~g의 일곱
 획이 모두 켜지고 dp만 꺼진다. 출력은 FE
 이며 교안의 값과 85ns 값과 일치한다. 1
 0~20ns에는 1의 두 획 b, c만 켜져 60이다.
 모든 구간에서 최하위 비트 seg_data[0]이
 0인지도 확인한다.

보드에서 확인할 입력과 출력

DIP1~4=bcd이며 단일 7세그먼트의 a~dp에 출력을 연결한다. 0, 1, 8, F를

입력하여 각각의 획이 예상대로 커지는지 확인한다. 입력 A~F의 표시도 검사하고, dp는 모든 입력에서 꺼져 있어야 한다.

참고문헌

- 전전설 교안 LAB1
- https://github.com/Glaysia/ece2_26_2_2/tree/main/example/fpga_projects_hdl/LAB1
- <https://docs.amd.com/r/en-US/ug835-vivado-tcl-commands/Introduction>
- FPGA관련 참고 문헌;
<https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/developer-guide/2024-2/fpga-architecture-overview.html>
- ASIC관련 참고 문헌;
https://www.arm.com/glossary/asic?utm_source